

Proxy inverso y balanceador de carga

Apache, Nginx y HAProxy

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 SRI · Servicios de Red e Internet

01

Proxy inverso

Concepto y casos de uso

¿Qué es un proxy inverso?

*" Un **proxy inverso** es un servidor que **acepta peticiones de los clientes**, las **reenvía a uno o varios servidores internos** y devuelve sus respuestas como si las hubiera generado él mismo.*

IDEA FUNDAMENTAL

- El cliente **no accede directamente** a los servidores backend
- El proxy es la **única cara visible** del servicio en Internet
- Los servidores internos pueden estar en una **red privada** sin acceso desde fuera
- Permite **centralizar** funcionalidades comunes (TLS, autenticación, caché...)

 Un **proxy directo** protege al cliente; un **proxy inverso** protege al servidor.

Casos de uso

PROTECCIÓN Y CENTRALIZACIÓN

- **Ocultar** la infraestructura backend
- Consolidar varias aplicaciones internas bajo un **único dominio público**
- Aplicar **filtros** y **autenticación** centralizada
- Centralizar la gestión de **cabeceras** de seguridad

RENDIMIENTO Y OPERACIÓN

- **Distribuir carga** entre varios backends
- **Terminación TLS**: HTTPS sólo en el proxy
- **Reescritura** de URLs y rutas internas
- **Caché** de respuestas estáticas

02

Proxy inverso con Apache

mod_proxy y ProxyPass

Activar los módulos necesarios

Apache implementa el proxy inverso con los módulos `mod_proxy` y `mod_proxy_http`.

```
sudo a2enmod proxy proxy_http
sudo systemctl reload apache2
```

OTROS MÓDULOS RELACIONADOS

- `mod_headers` — manipular cabeceras de la petición y la respuesta
- `mod_proxy_balancer` — balanceo de carga
- `mod_ssl` — terminación HTTPS en el propio proxy
- `mod_proxy_http2` — proxy a backends que hablan HTTP/2

Configuración básica

VirtualHost que reenvía **todas** las peticiones a un backend interno:

```
<VirtualHost *:80>
  ServerName proxy.example.org

  ProxyPass          "/" "http://interno.example.org/"
  ProxyPassReverse   "/" "http://interno.example.org/"

  ErrorLog  ${APACHE_LOG_DIR}/proxy_error.log
  CustomLog ${APACHE_LOG_DIR}/proxy_access.log combined
</VirtualHost>
```

- **ProxyPass** — reenvía las peticiones entrantes al backend
- **ProxyPassReverse** — reescribe en la respuesta cabeceras como **Location** para que apunten a la URL pública

El problema de las redirecciones

Cuando el backend responde con una redirección (por ejemplo a `/login`), suele incluir su **URL interna** en la cabecera `Location`:

```
Location: http://interno.example.org/login
```

Si esa URL llega tal cual al cliente, **no podrá resolverla** desde Internet.

 `ProxyPassReverse` reescribe automáticamente esa cabecera para que apunte al dominio público. Es **imprescindible** en cualquier proxy inverso.

Proxy de subrutas

Permite **publicar varias aplicaciones** internas bajo un mismo dominio público:

```
ProxyPass          "/web/" "http://interno.example.org/"
ProxyPassReverse   "/web/" "http://interno.example.org/"

ProxyPass          "/api/" "http://api.example.org/"
ProxyPassReverse   "/api/" "http://api.example.org/"
```

- `/web/` y `/api/` se redirigen a backends distintos
- El cliente sólo ve **un dominio**: `proxy.example.org`
- Cada subruta puede tener sus propias cabeceras y reglas

Cabeceras hacia el backend

`mod_headers` permite que el backend conozca **datos del cliente original**:

```
ProxyPreserveHost On

RequestHeader set X-Real-IP      "%{REMOTE_ADDR}s"
RequestHeader add X-Forwarded-For "%{REMOTE_ADDR}s"
RequestHeader set X-Forwarded-Proto "http"
RequestHeader set X-Forwarded-Host "%{HTTP_HOST}s"
```

CABECERA	SIGNIFICADO
<code>Host</code>	Dominio que pidió el cliente (con <code>ProxyPreserveHost On</code>)
<code>X-Real-IP</code>	IP real del cliente
<code>X-Forwarded-For</code>	Cadena de proxies intermedios
<code>X-Forwarded-Proto</code>	Protocolo del cliente (<code>http</code> / <code>https</code>)

03

Proxy inverso con Nginx

proxy_pass y proxy_params

Configuración básica

Nginx incluye soporte de proxy inverso de forma **nativa**, sin módulos adicionales.

```
server {  
    listen 80;  
    server_name proxy.example.org;  
  
    location / {  
        proxy_pass http://interno.example.org/;  
        include proxy_params;  
    }  
  
    access_log /var/log/nginx/proxy_access.log;  
    error_log /var/log/nginx/proxy_error.log;  
}
```

- **proxy_pass** — destino al que reenviar la petición
- **include proxy_params** — fragmento estándar de Debian con cabeceras útiles

El archivo `proxy_params`

`/etc/nginx/proxy_params` reenvía al backend la información del cliente original:

```
proxy_set_header Host          $http_host;
proxy_set_header X-Real-IP     $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

VARIABLE	CONTENIDO
<code>\$http_host</code>	Dominio que pidió el cliente
<code>\$remote_addr</code>	IP del cliente
<code>\$proxy_add_x_forwarded_for</code>	Acumula la cadena de proxies
<code>\$scheme</code>	Protocolo usado (<code>http</code> / <code>https</code>)

Reescritura de redirecciones

Nginx no reescribe automáticamente las cabeceras `Location`. Hay que indicarlo con `proxy_redirect`:

```
location / {  
    proxy_pass      http://interno.example.org/;  
    proxy_redirect  http://interno.example.org/ /;  
    include         proxy_params;  
}
```

 Equivale al `ProxyPassReverse` de Apache: reescribe las cabeceras `Location` y `Refresh` para que apunten al dominio público.

Proxy de subrutas

```
server {  
    listen 80;  
    server_name proxy.example.org;  
  
    location /web/ {  
        proxy_pass http://interno.example.org/;  
        include proxy_params;  
    }  
  
    location /api/ {  
        proxy_pass http://api.example.org/;  
        include proxy_params;  
    }  
}
```

⚠ Cuidado con la **barra final** en `proxy_pass`: con `/` al final se elimina el prefijo coincidente; sin barra, se reenvía la URL completa.

Apache vs Nginx — proxy inverso

ASPECTO	APACHE	NGINX
Módulos requeridos	<code>mod_proxy</code> , <code>mod_proxy_http</code>	Nativo
Reenvío	<code>ProxyPass</code>	<code>proxy_pass</code>
Reescritura de redirecciones	<code>ProxyPassReverse</code>	<code>proxy_redirect</code>
Cabeceras estándar	<code>mod_headers</code> (manual)	<code>proxy_params</code> (predefinido)
Conservar <code>Host</code>	<code>ProxyPreserveHost On</code>	<code>proxy_set_header Host \$http_host</code>

04

Balanceador de carga con HAProxy

Distribución de tráfico entre backends

¿Qué es HAProxy?

" *HAProxy (High Availability Proxy) es **software libre de alto rendimiento** especializado en **balanceo de carga** y **proxy inverso** para servicios basados en TCP y HTTP(S).*

POR QUÉ SE USA TANTO EN PRODUCCIÓN

- **Estabilidad** y rendimiento muy contrastados
- **Capacidades de monitorización** muy completas
- **Health checks** automáticos sobre los backends
- Soporta múltiples **algoritmos** de balanceo
- Puede balancear **TCP genérico** o entender **HTTP** y enrutar por cabeceras

Escenario de laboratorio

MÁQUINA	ROL	SERVICIO	IP
frontend	Balanceador	HAProxy	10.10.10.10
backend1	Servidor web	Apache / Nginx	10.10.10.11
backend2	Servidor web	Apache / Nginx	10.10.10.12



El cliente accede a `www.example.org` (la IP del frontend); HAProxy reparte las peticiones entre los dos backends de forma transparente.

Instalación

Instalación estándar en Debian 13:

```
sudo apt update  
sudo apt install haproxy
```

COMPROBACIONES

```
systemctl status haproxy  
  
# Validar la sintaxis antes de aplicar cambios  
sudo haproxy -c -f /etc/haproxy/haproxy.cfg  
  
# Aplicar cambios  
sudo systemctl restart haproxy
```

Estructura del archivo de configuración

`/etc/haproxy/haproxy.cfg` se organiza en secciones:

SECCIÓN	PARA QUÉ SIRVE
<code>global</code>	Parámetros del demonio (logs, usuario, límites, socket admin)
<code>defaults</code>	Valores por defecto para <code>frontend</code> y <code>backend</code>
<code>frontend</code>	Cómo se reciben las peticiones (puertos, ACLs)
<code>backend</code>	A qué servidores se reenvían y con qué algoritmo
<code>listen</code>	Combina <code>frontend</code> + <code>backend</code> en un solo bloque (más simple)

Sección `global` y `defaults`

```
global
  log /dev/log local0
  user haproxy
  group haproxy
  maxconn 2048
  stats socket /run/haproxy/admin.sock mode 660 level admin

defaults
  log      global
  mode     http
  option   httplog
  timeout  connect 5s
  timeout  client  30s
  timeout  server  30s
  retries  3
  maxconn  1024
```

- `mode http` — entiende HTTP (alternativa: `mode tcp` para TCP genérico)

Sección frontend

Define **cómo se reciben** las peticiones:

```
frontend servidores_web
  bind *:80
  default_backend servidores_web_backend

  # Página de estadísticas
  stats enable
  stats uri    /ha_stats
  stats refresh 10s
  stats auth   admin:admin123
```

- **bind *:80** — escucha en todas las interfaces, puerto 80
- **default_backend** — backend al que se mandan las peticiones por defecto
- **stats** — habilita la página de monitorización en **/ha_stats**

Sección backend

Define **a qué servidores** se reparten las peticiones:

```
backend servidores_web_backend
    balance roundrobin

    option      httpchk GET /
    http-check  expect status 200

    server backend1 10.10.10.11:80 check
    server backend2 10.10.10.12:80 check
```

- **balance** — algoritmo de balanceo
- **option httpchk** + **http-check expect** — comprueba periódicamente que el backend responde con 200
- **server ... check** — servidor con *health check* activo

Algoritmos de balanceo

ESTÁTICOS

- **roundrobin** — alterna las peticiones entre los servidores
- **static-rr** — round-robin sin tener en cuenta los pesos dinámicos
- **source** — el mismo cliente (IP) va siempre al mismo servidor (afinidad)
- **uri** — distribuye según un *hash* de la URI

DINÁMICOS

- **leastconn** — al servidor con **menos conexiones** activas
- **first** — al primero disponible (escalado bajo demanda)
- **random** — elección aleatoria con distribución uniforme

Health checks

HAProxy comprueba periódicamente que cada backend está **operativo** y deja de enviarle tráfico si falla.

```
backend servidores_web_backend
  option      httpchk GET /healthz
  http-check expect status 200

server backend1 10.10.10.11:80 check inter 2s fall 3 rise 2
server backend2 10.10.10.12:80 check inter 2s fall 3 rise 2
```

PARÁMETRO	SIGNIFICADO
inter	Intervalo entre comprobaciones
fall	Comprobaciones fallidas seguidas para marcarlo caído
rise	Comprobaciones correctas seguidas para volver a marcarlo activo

Página de estadísticas

Una vez aplicada la configuración, basta acceder a:

```
http://www.example.org/ha_stats
```

con las credenciales `admin` / `admin123`.

INFORMACIÓN QUE MUESTRA

- Estado de cada **backend** (UP / DOWN / MAINT)
- **Conexiones activas** y totales por servidor
- Tiempos de respuesta y métricas de error
- Permite **deshabilitar manualmente** un nodo (mantenimiento)



La página de estadísticas **nunca** debe quedar accesible sin autenticación ni en redes públicas: muestra detalles

Acceso desde el cliente

Para probar el balanceo, añadir la resolución estática en `/etc/hosts`:

```
10.10.10.10    www.example.org
```

A continuación, recargar varias veces:

```
http://www.example.org/
```

Las respuestas alternarán entre `backend1` y `backend2` (con `roundrobin`).

 Si los backends sirven páginas distintas (con su nombre), se ve a simple vista cómo HAProxy reparte las peticiones.

Monitorización con HATop

HATop es una herramienta de **terminal** que muestra en tiempo real el estado de HAProxy.

```
sudo apt install hatop  
sudo hatop -s /run/haproxy/admin.sock
```

ATAJOS ÚTILES

- **F9** — habilitar un servidor
- **F10** — deshabilitar un servidor (mantenimiento)
- Navegación entre frontends y backends con cursores



Útil para operar el balanceador desde la consola sin abrir un navegador en la red interna.

Para profundizar

- [Documentación oficial de HAProxy](#)
- [Apache mod_proxy](#)
- [Nginx — proxy_pass](#)

SRI · UNIDAD 3 — PROXY INVERSO Y BALANCEADOR DE CARGA

¡Gracias!

Proxy inverso y balanceo → Alta disponibilidad

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 SRI